

Mestrado em Engenharia Informática – Cibersegurança e Administração de Sistemas

Phase 1 – Analysis Stage

DESOFS

1250495 António Neves
1250498 Beatriz Ferreira
1201264 Francisco Gouveia
1151352 João Mata

Contents

Introduction.....	4
Project Overview.....	4
Application Description.....	4
Role Definition.....	4
Key Functional Requirements Summary	5
System Architecture Analysis.....	6
Architectural Overview.....	6
Domain Model (DDD) Security Analysis.....	7
Main Security Considerations.....	7
Data Flow Diagrams	9
Level 0	9
Level 1	9
5. Threat Modeling and Abuse Cases	10
5.1 Threat Modeling Approach	10
5.2 Scope, Trust Boundaries, and Data Flows.....	10
5.3 STRIDE Analysis by DFD Element.....	12
External Entity: Client.....	12
Process: AIM Middleware and REST API Controllers	12
Process: Application Services.....	12
Data Store: Relational Database.....	13
Process / External Interaction: Payment API Client and External Payment Provider	13
Process / Data Store: OS File Adapter and File System Utilities	13
5.4 Abuse Cases.....	15
AC-01 – Brute-Force Authentication Attack.....	16
AC-02 – Direct Access to Restricted Endpoints	16
AC-03 – Concurrent Order Abuse to Break Stock Consistency	16
AC-04 – Malicious File Upload via Tea Image Management.....	16
AC-05 – Payment State Forgery	16
AC-06 – Sensitive Data Enumeration Through API Errors	16
AC-07 – Sales Report Abuse	16
AC-08 – API Flooding	17
5.5 Abuse Case Diagrams.....	17
5.6 Traceability Between Threats, Abuse Cases, and Risks.....	19
Risk Assessment and Mitigation	20
Risk Assessment and Methodology.....	20
Critical Assets	20
Identified Risks.....	21
R1 - Authentication Bypass.....	21

R2 - SQL Injection	22
R3 - Concurrency Issues (Race Condition in Stock Allocation)	22
R4 - Malicious File Upload	23
R5 - Payment Status Manipulation.....	23
R6 - Exposure of Sensitive Data	24
R7 - Denial of Service (DoS).....	24
R8 – Broken Access Control / Privilege Escalation	25
R9 – OS Command Injection / Path Traversal.....	25
Security requirements	26
ASVS Verification Level.....	26
Fully applicable chapters	27
Partially applicable chapters	27
Non-applicable chapters	27
Requirements Mapping by Aggregate	28
User Aggregate.....	28
Session Aggregate	28
Testing Strategy.....	29
Testing Strategy	29
Test Mapping	30
Tools and Pipeline Integration	30
Test Priorities	31

Introduction

This report documents the work carried out during Sprint 1 of the DESOFS project, developed as part of the Master's in Computer Engineering at ISEP for the 2025/2026 academic year. The goal of this curricular unit is to develop a software system following the Secure Software Development Life Cycle (SSDLC), where security is not an afterthought but a discipline embedded throughout every stage of the project.

Sprint 1 focused on analysis and design. The team defined the system's requirements, established its architecture, and modelled its domain. All of these activities were conducted with a strong security bias - architectural decisions were evaluated for their security implications, the domain model was examined for sensitive data and dangerous operations, and the OWASP Application Security Verification Standard (ASVS) version 5.0.0 was adopted as the compliance framework that will guide the project through all subsequent sprints.

Project Overview

Application Description

The Tea Shop application is a web-based e-commerce platform that allows customers to browse and purchase teas, while managers and administrators oversee inventory and operational concerns. The system exposes a RESTful API as its backend, built upon a layered, Domain-Driven Design (DDD) architecture.

The application manages five core domain aggregates: Users (IAM), Teas (product catalog), Orders, Payments, and Sessions. It integrates with an External Payment Provider and leverages the server's OS file system for image storage, both of which introduce external trust boundaries that require careful security treatment.

Role Definition

The system defines three distinct user roles, each with a distinct authorization scope:

Role	Description	Key Permissions
CUSTOMER	Registered end-users of the platform	Browse catalog, place orders, payment process
MANAGER	Operational staff	All CUSTOMER permissions, plus: create/update/delete teas, upload/delete product images, update order status
ADMIN	System administrators	All MANAGER permissions, plus: request sales report exports with custom parameters

Key Functional Requirements Summary

The following table summarises the functional requirements and flags those with direct security implications:

ID	Requirement	Security Relevance
FR-01	User registration (email + password)	Password policy, credential storage, input validation
FR-02	User authentication / session token issuance	Credential brute-force, session fixation, token security
FR-03	Role assignment (CUSTOMER, MANAGER, ADMIN)	Privilege escalation, broken access control
FR-04	Address management (authenticated)	Authorization enforcement
FR-05	Browse tea catalog (all users)	Information disclosure, public endpoint exposure
FR-06	Inventory management (MANAGER/ADMIN)	Authorization, input validation, injection
FR-07	Tea image upload/delete via OS file system	Path traversal, OS command injection, arbitrary file upload
FR-08	Place order (CUSTOMER)	Business logic bypass, IDOR on order resources
FR-09	Automatic stock deduction on order	Race condition, negative stock manipulation
FR-10	Order status update (MANAGER)	unauthorized status transitions
FR-11	Payment processing	Payment tampering, replay attacks, third-party trust
FR-13	Sales report export with custom Report Name (ADMIN)	OS command injection via report name, path traversal, injection

System Architecture Analysis

Architectural Overview

The system follows a four-layer architecture, as illustrated in Figure 1. All client HTTP requests first pass through the AIM Middleware in the Presentation Layer, which is responsible for token validation before any request reaches the REST API Controllers. Validated requests are then forwarded as DTOs to the Application Layer, where the Application Services coordinate business logic, delegating to the Domain Model and to three infrastructure adapters: the Database Repository, the OS File Adapter, and the Payment API Client.

Two external dependencies exist outside the system boundary: a Relational Database accessed via SQL, and a Server Operating System whose File System Utilities are used for image storage. A third external dependency is the External Payment Provider, which is reached via outbound API calls from the Payment API Client.

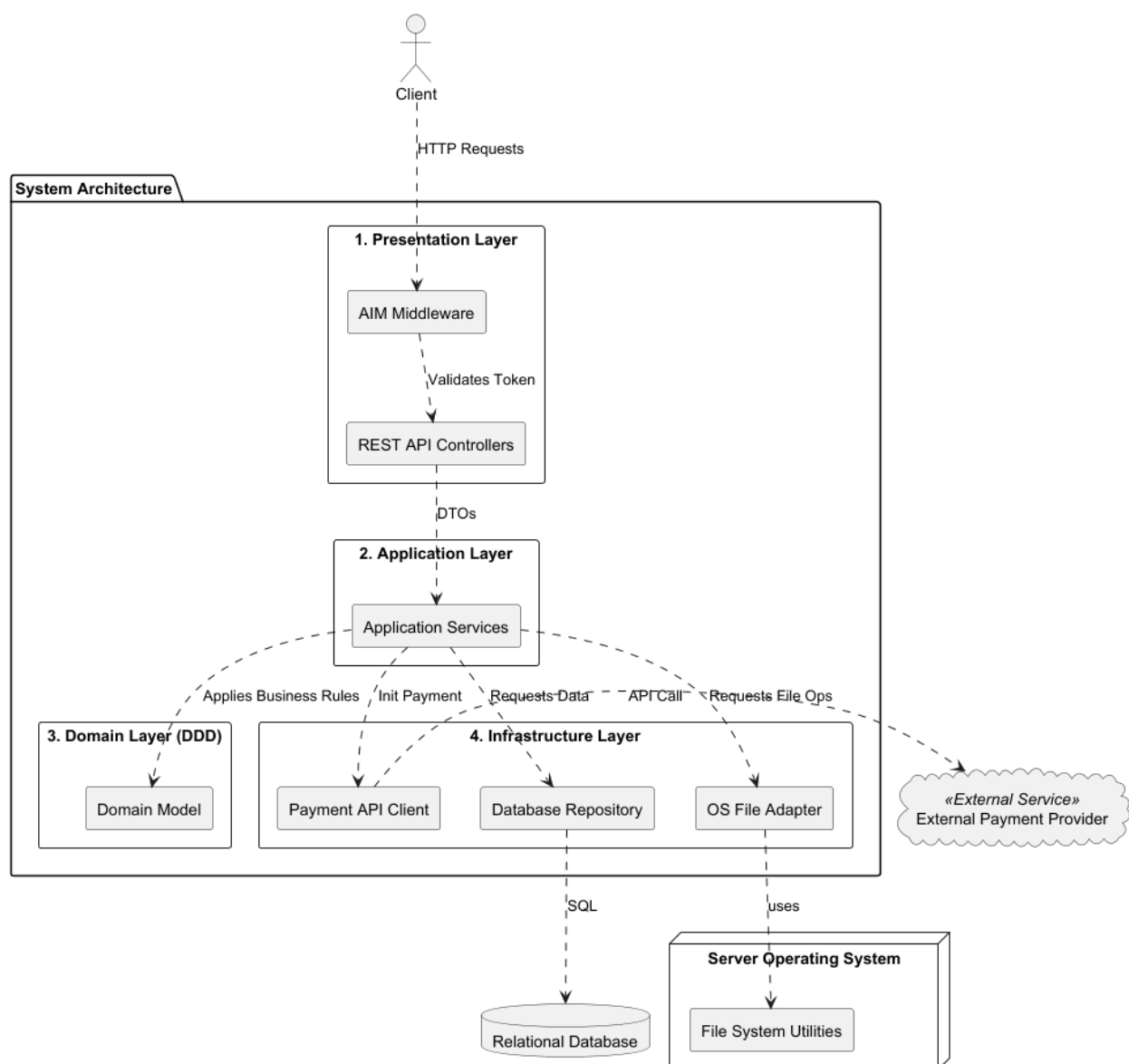


Figure 1: System Architecture

Domain Model (DDD) Security Analysis

The domain is modelled following Domain-Driven Design (DDD) principles and is organised into five aggregates, as shown in Figure 2. Each aggregate encapsulates its own consistency boundary and exposes behaviour through its aggregate root.

- **IAM Aggregate** - manages user sessions through the Session root, holding the session token, its validity period, and a revocation flag.
- **User Aggregate** - represents registered users, their credentials (stored as a password hash), their role (CUSTOMER, MANAGER, or ADMIN), and their associated addresses.
- **Tea Aggregate** - represents the product catalog. The Tea root holds pricing and stock level information and exposes operations for uploading and deleting images, which are stored via the OS File Adapter.
- **Order Aggregate** - manages the order lifecycle through a defined set of statuses (PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED) and contains line items referencing Tea entries. It also includes the SalesReport entity, which is generated on demand by administrators.
- **Payment Aggregate** - tracks the payment associated with an order, including its status (PENDING, COMPLETED, FAILED, REFUNDED) and the external transaction identifier returned by the payment provider.

Main Security Considerations

The following table analyses each domain aggregate from a security perspective, identifying sensitive data, dangerous operations, and checks that must be enforced throughout implementation.

Aggregate	Sensitive Data	Dangerous Operations	Key Invariants
IAM	sessionToken	Session.isValid() — central enforcement point for all authenticated requests	isRevoked overrides expiry. Token entropy must resist enumeration.
User	passwordHash, role, email, address (GDPR)	Role assignment — drives all downstream authorization decisions	Role changes: ADMIN only. passwordHash never leaves the persistence layer.
Tea	stockLevel, pricing	uploadImage / deleteImage — only point where user input reaches an OS-level interface	Writes: MANAGER/ADMIN only. File name and type validated before reaching OS adapter. stockLevel requires concurrency control.
Order	Order history, line items, delivery address (user-scoped)	generateSalesReport — user-controlled name flows into a file path	Reads/writes scoped to owner. Stock deduction atomic with order creation.
Payment	payment status, transactionId (no card data stored)	external input drives payment status	Status once COMPLETED or FAILED has to be immutable.

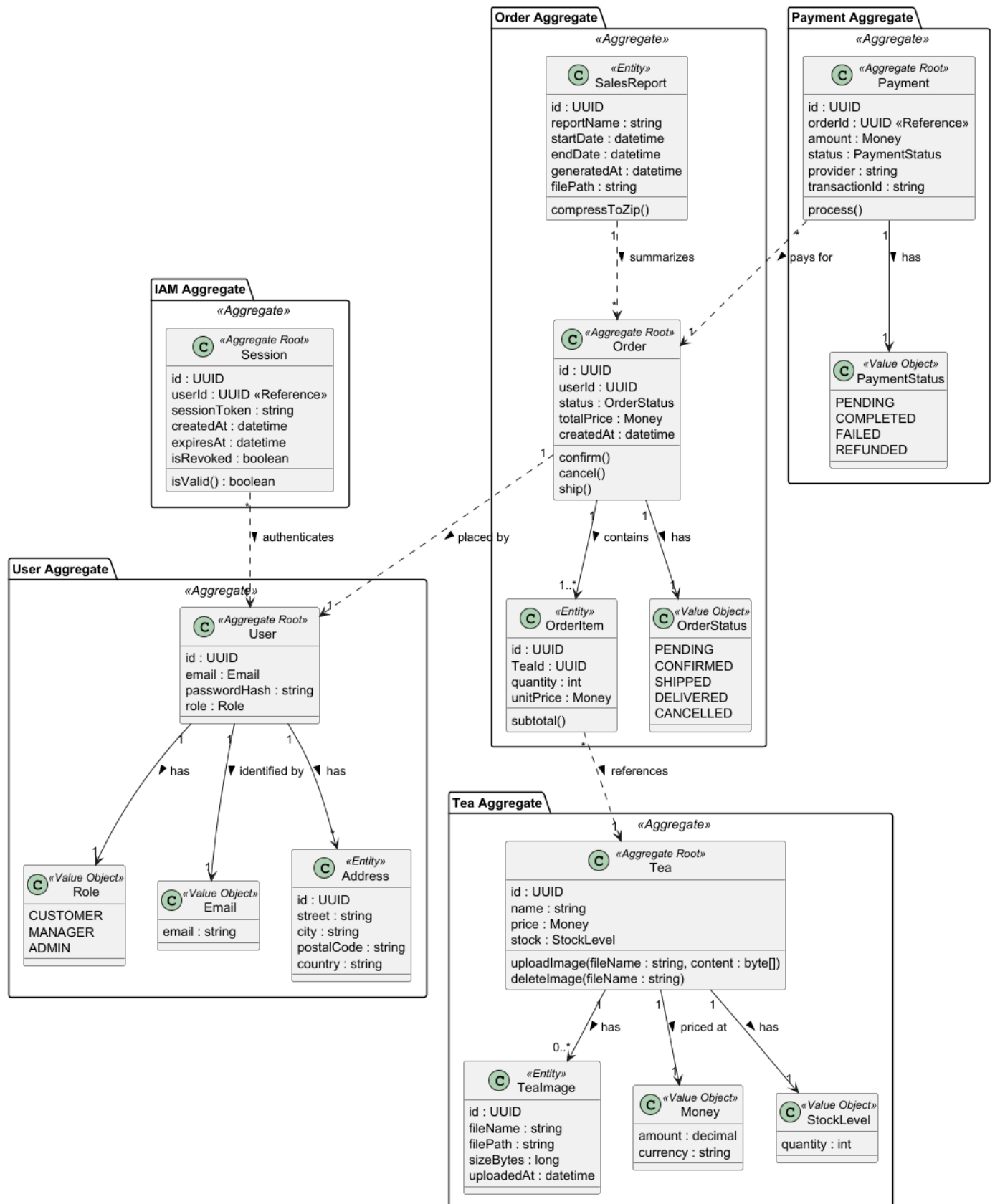


Figure 2:DDD Model

Data Flow Diagrams

As a representation of the data flows between the components of the system, the following 2 Data Flow diagrams were created, displaying the system with different levels of detail.

Level 0

The Level 0 diagram displays the system and its interactions with external entities.

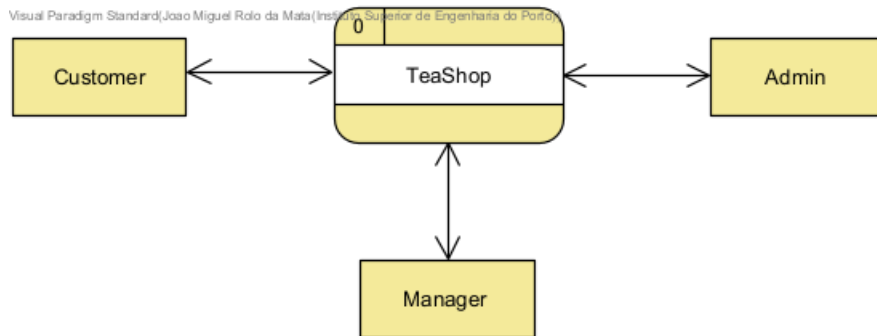


Figure 3 - Level 0 Data Flow Diagram - Context view

Level 1

The Level 1 diagram presented below represents the processes of the system.

The following processes are represented:

- FR-01 - Customer Registration
- FR-02 - Customer authentication
- FR-04 - Update Shipping Address
- FR-08 - Place Order
- FR-06 - Create Tea Entries
- FR-07 - Modify Tea images
- FR-10 - Update status of order
- FR-13 - Request sales report

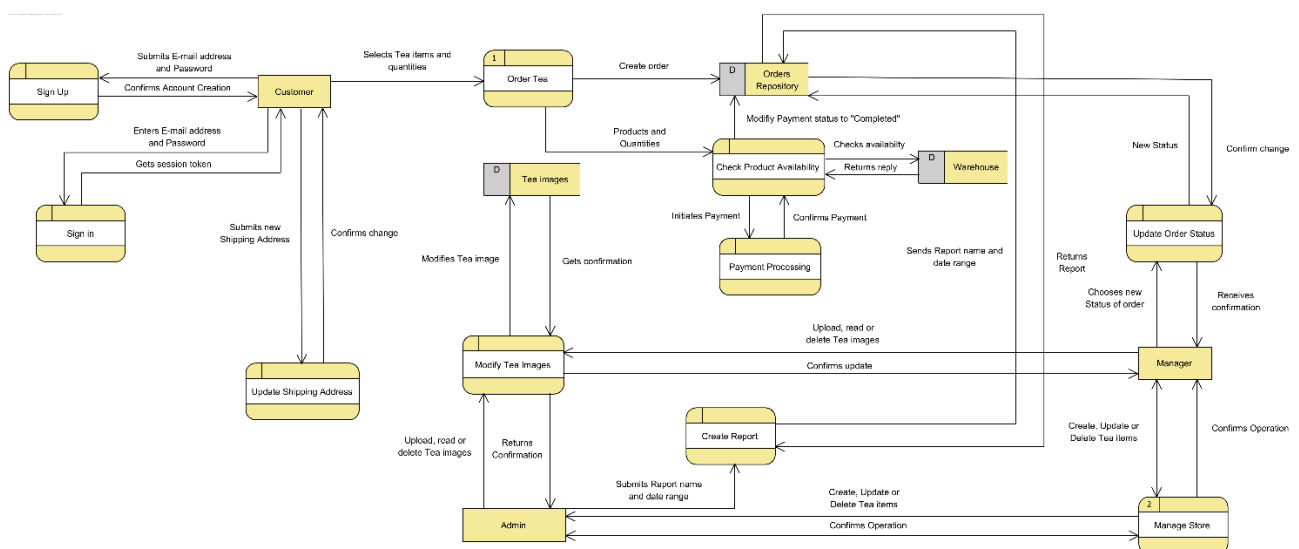


Figure 4 - Level 1 DFD

5. Threat Modeling and Abuse Cases

5.1 Threat Modeling Approach

The threat model for the Tea Shop backend was developed using the **STRIDE** methodology. This approach was selected because it provides a structured way to identify threats affecting the main system elements, including external entities, internal processes, data flows, and data stores. It is also consistent with the objectives of Phase 1, where threat modeling and abuse case analysis are explicitly required as part of the SSDLC analysis and design activities.

The analysis was based on the system requirements, the layered architecture, and the Domain-Driven Design model previously defined by the team. In particular, attention was given to the most security-sensitive capabilities of the application, namely authentication, role-based authorization, stock allocation, payment processing, report generation, and image management through backend operating system interaction. These elements are directly reflected in the project requirements and in the architecture and domain diagrams already presented in this report.

Rather than treating threats only at a generic level, the model was explicitly aligned with the system's architectural flows and trust boundaries. This makes it possible to show not only which threats exist, but also where they materialize in the application and which components are directly exposed.

In addition to the architectural and domain views, the threat model was also informed by the system's data-flow representations, which helped identify external actors, core processes, and relevant trust boundaries.

5.2 Scope, Trust Boundaries, and Data Flows

From the architectural perspective, the most security-relevant elements of the system are the **Client**, the **AIM Middleware**, the **REST API Controllers**, the **Application Services**, the **Database Repository**, the **OS File Adapter**, the **Payment API Client**, the **Relational Database**, the **Server File System Utilities**, and the **External Payment Provider**. These components are directly involved in the main trust boundaries of the platform and either process untrusted input, execute privileged operations, or store critical data. The architectural overview presented earlier already shows these interactions in a DFD-oriented manner.

From the domain perspective, the analysis focuses on the **IAM Aggregate**, **User Aggregate**, **Tea Aggregate**, **Order Aggregate**, and **Payment Aggregate**. These aggregates encapsulate assets such as credentials, session tokens, addresses, tea stock, orders, payment state, and sales-report-related data. Their compromise would directly affect confidentiality, integrity, availability, or business correctness. The DDD analysis already identifies these aggregates and their main security considerations, especially for session validation, role assignment, stock consistency, file handling, and payment status control.

From a broader system-analysis perspective, the threat model also takes into account the main operational flows of the application, particularly those related to registration, address management, order placement, store management, and order status updates. These flows help identify where user-controlled input enters the system, where privileged actions are performed, and where business-critical data is created or modified.

The main trust boundaries identified in the system are the following:

Trust Boundary	Description	Main Security Concern
External Client -> Trusted Backend	Requests enter through AIM Middleware and REST API Controllers	Untrusted input, spoofing, brute force, DoS
Backend -> Relational Database	Application Services persist and retrieve business data	Integrity, confidentiality, injection, transactional consistency
Backend -> Server File System	Image operations are executed through the OS File Adapter	Arbitrary file upload, path abuse, unsafe file handling
Backend -> External Payment Provider	Payment operations cross into an external service	Trust validation, replay, tampering, response integrity
Unauthenticated -> Authenticated / Privileged Sessions	Access depends on valid session tokens and role checks	Broken authentication, broken access control

The most relevant data flows for the threat model are:

- **Client -> AIM Middleware -> REST API Controllers**
- **Application Services -> Database Repository -> Relational Database**
- **Application Services -> OS File Adapter -> File System Utilities**
- **Application Services -> Payment API Client -> External Payment Provider**

These flows were used as the main basis for threat identification.

5.3 STRIDE Analysis by DFD Element

To improve traceability, the STRIDE analysis was performed by associating threats with concrete DFD-oriented elements of the system.

External Entity: Client

The **Client** is outside the system boundary and must therefore be treated as untrusted. Through this entity, attackers may attempt **spoofing** by using stolen credentials or replayed session tokens, **tampering** by manipulating request payloads, **repudiation** by denying previous actions, **information disclosure** by probing API behaviour, **denial of service** by flooding endpoints, and **elevation of privilege** by attempting direct access to restricted functionality.

This element is particularly relevant to the authentication flow, catalog access, order placement, payment initiation, image upload, and report export.

Process: AIM Middleware and REST API Controllers

The **AIM Middleware** and **REST API Controllers** are directly exposed to external input and form the first defensive layer of the application. Weak token validation may enable **spoofing**; insufficient validation of requests may allow **tampering**; verbose messages may enable **information disclosure**; request flooding may result in **denial of service**; and incomplete access checks may enable **elevation of privilege**.

Since the client is not a trusted boundary, all security-relevant input validation must be enforced on the backend. Client-side validation may improve usability, but it cannot be relied upon as a security control.

Process: Application Services

The **Application Services** orchestrate the system's most important business operations, including authentication, stock deduction, payment processing, report generation, and privileged management actions. As such, they are central to the integrity of the platform.

At this level, the most relevant threats are **tampering** with business logic, **repudiation** of privileged actions, **information disclosure** through insecure responses, **denial of service** through repeated expensive operations, and **elevation of privilege** if authorization checks are not enforced consistently at service level.

Report generation is also a relevant tampering and information disclosure surface, especially when user-controlled parameters such as `reportName` may influence file creation or file path resolution.

Data Store: Relational Database

The **Relational Database** stores users, sessions, teas, stock levels, orders, payments, and report-related information. It is therefore a critical data store from both security and business perspectives.

This element is mainly exposed to **tampering**, especially if upstream controls fail and malicious input reaches persistence logic; to **information disclosure** if access restrictions are weak; and to **denial of service** if expensive queries or repeated transactional conflicts reduce responsiveness.

Process / External Interaction: Payment API Client and External Payment Provider

The payment integration crosses a trust boundary between the internal backend and an external service. This makes it particularly relevant for **spoofing**, **tampering**, and **repudiation** threats.

An attacker may attempt to manipulate payment initiation or response handling in order to force inconsistent payment state transitions. Since payment processing is one of the project's core business requirements, this interaction must be considered one of the most security-sensitive flows of the system.

Process / Data Store: OS File Adapter and File System Utilities

The **OS File Adapter** and the underlying server file system are security-sensitive because the platform supports tea image upload, read, and deletion through backend operating system interaction. This directly reflects the requirement for image management with OS integration.

This part of the architecture is mainly exposed to **tampering** through malicious uploads, **information disclosure** through predictable or unsafe file access patterns, **denial of service** through large or repeated uploads, and potentially **elevation of privilege** if file handling is implemented insecurely.

DFD Process	Main STRIDE Concerns	Reason
Sign Up	Spoofing, Tampering, DoS	Accepts untrusted registration input and credential submission
Update Shipping Address	Tampering, Elevation of Privilege, Information Disclosure	Updates user-related data and must be scoped to the authenticated owner
Order Tea	Tampering, DoS	Creates orders and may be abused through malformed or concurrent requests
Manage Store	Elevation of Privilege, Tampering	Allows privileged inventory operations
Update Order Status	Tampering, Elevation of Privilege, Repudiation	Changes business-critical order state and must be attributable
Order (Data Store)	Tampering, Information Disclosure	Stores order details and order status

Table 1 - STRIDE summary by DFD-oriented element

5.4 Abuse Cases

To complement the threat model, a set of abuse cases was defined. While the STRIDE analysis is organised by threat category and system element, abuse cases describe how malicious actors may intentionally misuse legitimate functionality to achieve harmful goals. This makes them easier to relate to future security testing and to the risk assessment already produced by the team.

The identified abuse cases are the following:

ID	Abuse Case	Threat Actor	Main Target
AC-01	Brute-Force Authentication Attack	External attacker	Authentication flow
AC-02	Direct Access to Restricted Endpoints	Malicious authenticated customer	Admin/manager operations
AC-03	Concurrent Order Abuse to Break Stock Consistency	Authenticated customer	Stock allocation flow
AC-04	Malicious File Upload via Tea Image Management	Malicious or compromised privileged user	File handling flow
AC-05	Payment State Forgery	Customer or API attacker	Payment flow
AC-06	Sensitive Data Enumeration Through API Errors	External attacker	REST API responses
AC-07	Sales Report Abuse via 'reportName' Manipulation	Low-privilege authenticated user	Report generation and file handling flow
AC-08	API Flooding	External attacker or bot	Public and sensitive endpoints

Table 2 - Abuse Cases

The most relevant abuse scenarios are described below.

AC-01 – Brute-Force Authentication Attack

An external attacker may repeatedly submit authentication requests using guessed credentials in an attempt to discover valid accounts or gain unauthorized access. This abuse case targets the flow between the **Client**, the **AIM Middleware**, the **REST API Controllers**, and the session or user authentication logic. Its expected impact includes account compromise, unauthorized access, and possible privilege escalation.

AC-02 – Direct Access to Restricted Endpoints

A malicious authenticated customer may attempt to bypass interface restrictions and directly invoke endpoints intended only for MANAGER or ADMIN, such as inventory management, order status updates, and sales report export. Its impact includes unauthorized execution of privileged operations and access to sensitive business data.

AC-03 – Concurrent Order Abuse to Break Stock Consistency

An authenticated attacker may intentionally send multiple concurrent order requests for the same tea item when stock is low, attempting to exploit race conditions in the stock allocation process. Its impact includes negative stock values, overselling, data inconsistency, and operational disruption. This abuse case is directly linked to the stock allocation requirement and to the need for safe concurrency handling and transactional integrity.

AC-04 – Malicious File Upload via Tea Image Management

A malicious or compromised privileged user may upload a file disguised as a tea image in order to exploit unsafe file handling on the backend. Its impact includes malicious file persistence, operating system abuse, and possible server compromise.

AC-05 – Payment State Forgery

An attacker may attempt to manipulate the payment flow in order to force an order into a successful payment state without a legitimate transaction. Its impact includes fraudulent purchases, direct financial loss, and compromise of transaction integrity. This abuse case is directly linked to **FR-11 (Payment Processing)** and to the trust boundary with the external payment provider.

AC-06 – Sensitive Data Enumeration Through API Errors

An attacker may intentionally submit malformed or unauthorized requests and compare API responses or behavioural differences in order to infer valid accounts, internal identifiers, or system structure. Its impact includes information disclosure and support for subsequent attacks.

AC-07 – Sales Report Abuse

A low-privilege or unauthorized user may attempt to abuse the sales report export functionality by manipulating report-related parameters, especially the custom `reportName`, in order to access sensitive business information or influence backend file handling.

This abuse case is particularly relevant because the domain model includes a `SalesReport` entity with attributes such as `reportName` and `filePath`, which suggests that user-controlled input may influence the generation and storage of exported report files. If the report name is not properly validated, sanitized, and normalized before being used in file-related operations, an attacker may attempt path traversal or other forms of file-path abuse.

Its impact includes unauthorized disclosure of commercially sensitive information, unsafe interaction with the file system, and possible compromise of report export integrity.

AC-08 – API Flooding

An attacker may flood authentication, catalog, order, upload, or report-related endpoints with a large number of requests in order to degrade performance or exhaust backend resources. Its impact includes increased latency, degraded availability, and denial of service.

5.5 Abuse Case Diagrams

To make the misuse scenarios more explicit, selected abuse cases were also represented using UML abuse case diagrams. These diagrams visually show the relationship between legitimate use cases, malicious use cases, attackers, and mitigation-oriented controls. This visual representation improves readability and makes the connection between the architecture and the attacker goals easier to understand.

The most relevant abuse cases represented visually are:

- brute-force authentication;
- direct access to restricted endpoints;
- malicious file upload;
- payment state forgery.

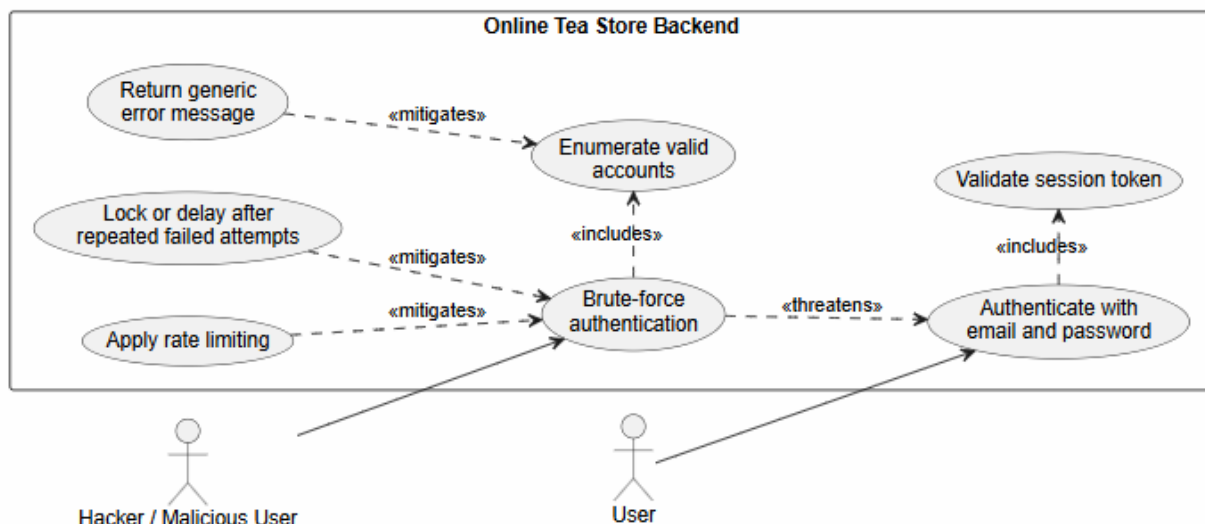


Figure 1 – Brute-force authentication

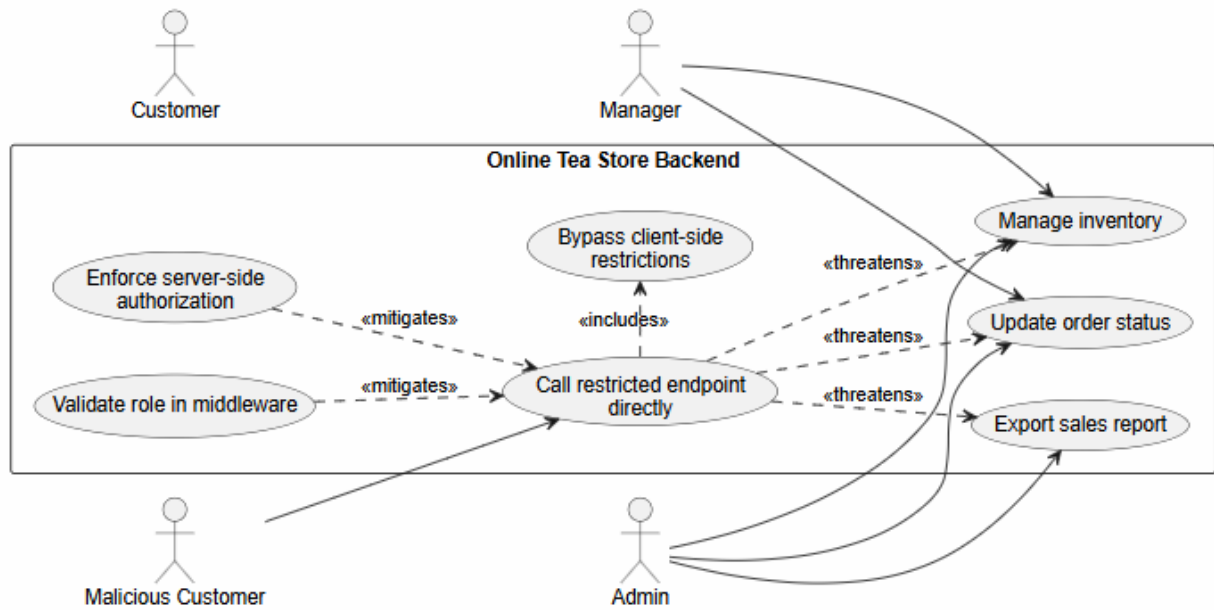


Figure 2 - Direct access to restricted endpoints

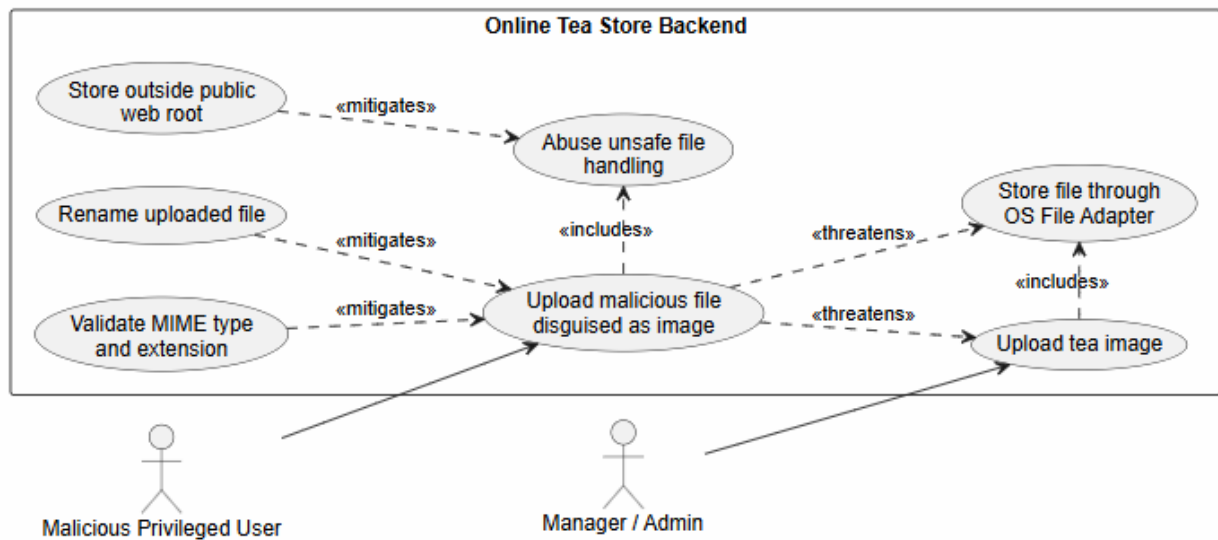


Figure 3 - Malicious file upload

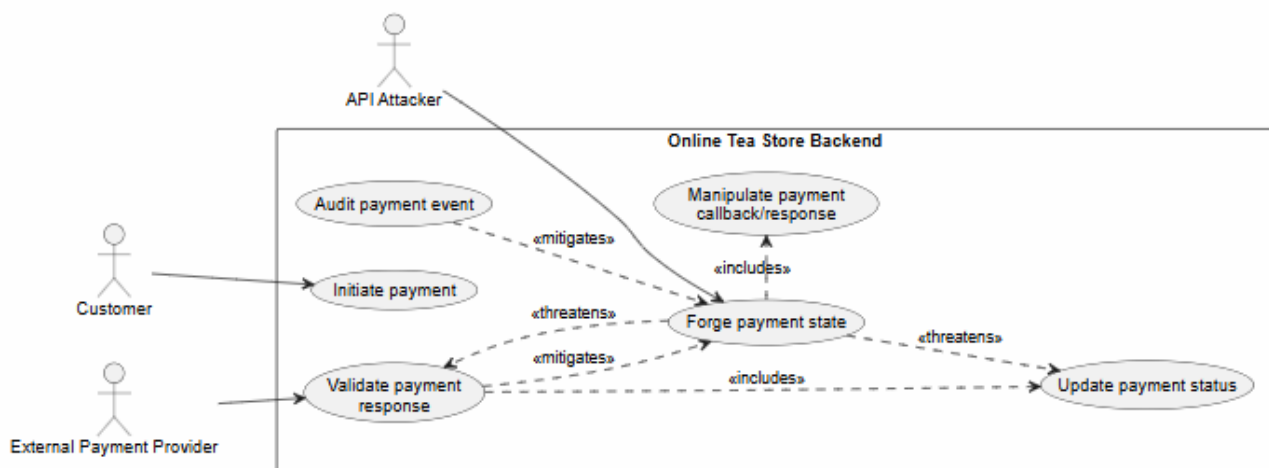


Figure 4 - Payment state forgery

5.6 Traceability Between Threats, Abuse Cases, and Risks

The abuse cases are directly derived from the identified threats and are consistent with the risks prioritised in the team's risk assessment. Authentication-related abuse cases support the analysis of authentication bypass and broken access control. Stock and payment abuse cases support integrity-related risks. File upload and report abuse support risks related to unsafe operating system interaction and unauthorized access to sensitive data. Finally, flooding abuse cases support denial-of-service analysis.

This traceability is important because it ensures that the threat model is not an isolated section of the report. Instead, it is directly connected to the requirements, the architecture, the domain model, the risk assessment, and the testing strategy already defined by the team.

Risk Assessment and Mitigation

Risk Assessment and Methodology

A qualitative risk assessment methodology was adopted based on the following formula: **Risk = Likelihood x Impact**. Each factor is evaluated on a three-level scale.

Level	Likelihood	Impact
Low	1	1
Medium	2	2
High	3	3

Risk levels are classified as:

- Low: 1 to 2;
- Medium: 3 to 4;
- High: 6 to 9.

This methodology allows prioritization of threats affecting critical system assets such as user credentials, financial transactions, stock management and file system operations,

Critical Assets

Based on defined Domain Model (DDD) and layered architecture, the following assets were identified as critical to the system's security operation:

- **User Credentials (User Aggregate)**
Includes email, passwordHash, and role (CUSTOMER, MANAGER, ADMIN).
This asset is critical as it controls authentication and authorization across the system.
- **Order Data (Order Aggregate)**
Contains information about user purchases, quantities, and order status.
Compromise of this data may lead to financial inconsistencies and loss of integrity.
- **Payment Information (Payment Aggregate)**
Represents payment status and transaction results.
This asset is highly sensitive as manipulation may result in financial fraud.
- **Tea Stock Levels (Tea Aggregate: StockLevel)**
Tracks product availability and is directly linked to business operations.
Incorrect values may lead to overselling or inconsistencies (violating NFR:02).
- **Relational Database (Infrastructure Layer)**
Stores all persistent data, including users, orders, and payments.
It is a central asset whose compromise affects the entire system.
- **File System (Tea Image Storage: OS Integration)**
Used to store uploaded tea images (FR07).
Improper handling may lead to malicious file execution or server compromise.

- **REST API Endpoints (Presentation Layer)**
Entry point for all client interactions with the system.
Vulnerabilities here may expose all underlying components.

Identified Risks

R1 - Authentication Bypass

Affected Components:

User Aggregate, REST API (Presentation Layer)

Related Requirements:

FR-02 (User authentication / session token issuance), FR-03 (Role assignment)

Description:

Improper authentication mechanisms or weak token validation may allow unauthorized users to gain access to privileged roles such as ADMIN or MANAGER.

Attacker Objective:

Gain unauthorized access to the system and escalate privileges to perform administrative actions.

Threat Impact:

Unauthorized access may lead to data breaches, reputational damage, and loss of customer trust. It may also allow manipulation of system operations by non-authorized users.

Likelihood: High (3)

Impact: High (3)

Risk Level: High (9)

R2 - SQL Injection

Affected Components:

Infrastructure Layer (Database Repository), Application Services

Related Requirements:

FR-06 (Inventory Management)

Description:

Improper validation or sanitization of user input in API requests may allow attackers to inject malicious SQL queries.

Attacker Objective:

Access, modify, or extract sensitive data from the database (users, orders, payments).

Threat Impact:

A successful attack may result in exposure of customer data, legal consequences (e.g., GDPR violations), financial penalties, and loss of business credibility.

Likelihood: High (3)

Impact: High (3)

Risk Level: High (9)

R3 - Concurrency Issues (Race Condition in Stock Allocation)

Affected Components:

Order Aggregate, Tea Aggregate (StockLevel)

Related Requirements:

FR-09 (Automatic stock deduction on order)

Description:

Simultaneous purchase requests for the same product may lead to inconsistent stock values if concurrency is not properly handled.

Attacker Objective:

Exploit race conditions to purchase more items than available or manipulate stock behavior.

Threat Impact:

Incorrect stock management may lead to overselling, order cancellations, customer dissatisfaction, and operational inefficiencies.

Likelihood: Medium (2)

Impact: High (3)

Risk Level: High (6)

R4 - Malicious File Upload

Affected Components:

Tea Aggregate (uploadImage), OS File Adapter (Infrastructure Layer)

Related Requirements:

FR-07 (Tea image upload/delete via OS file system)

Description:

Unrestricted file upload functionality may allow attackers to upload malicious or executable files to the server.

Attacker Objective:

Gain remote access to the server or execute malicious code.

Threat Impact:

Server compromise may result in full system downtime, data breaches, and significant recovery costs, impacting business continuity.

Likelihood: High (3)

Impact: High (3)

Risk Level: High (9)

R5 - Payment Status Manipulation

Affected Components:

Payment Aggregate

Related Requirements:

FR11 (Payment Processing)

Description:

An attacker may attempt to manipulate the payment status, for example by forcing a transaction to be marked as COMPLETED.

Attacker Objective:

Obtain products or services without completing a legitimate payment.

Threat Impact:

Fraudulent transactions may lead to direct financial loss, accounting inconsistencies, and loss of trust from customers and partners.

Likelihood: Medium (2)

Impact: High (3)

Risk Level: Medium–High (6)

R6 - Exposure of Sensitive Data

Affected Components:

REST API Responses, User Aggregate

Related Requirements:

FR-01 (User registration), FR-02 (User Authentication), FR-05 (Browse tea catalog)

Description:

Improper handling of API responses may expose sensitive information such as passwordHash or internal system data.

Attacker Objective:

Collect sensitive information to perform further attacks (e.g., credential reuse, privilege escalation).

Threat Impact:

Leakage of sensitive data may result in regulatory non-compliance (e.g., GDPR), reputational damage, and potential legal actions.

Likelihood: Medium (2)

Impact: High (3)

Risk Level: Medium–High (6)

R7 - Denial of Service (DoS)

Affected Components:

REST API (Presentation Layer)

Related Requirements:

FR-05 (Browse tea catalog)

Description:

Excessive or malicious requests may overload the system, leading to reduced performance or service unavailability.

Attacker Objective:

Disrupt system availability and degrade service for legitimate users.

Threat Impact:

Service unavailability may result in lost sales, degraded customer experience, and damage to the company's reputation.

Likelihood: Medium (2)

Impact: Medium (2)

Risk Level: Medium (4)

R8 – Broken Access Control / Privilege Escalation

Affected Components:

`User Aggregate`, `Application Layer`, `REST API`

Related Requirements:

FR-03 (Role assignment), FR-04 (Address management), FR-06 (Inventory management)

Description:

Improper enforcement of role-based access control may allow users to access or modify resources without proper authorization.

Attacker Objective:

Escalate privileges or access restricted functionalities.

Threat Impact:

Unauthorized actions may lead to data manipulation, system misuse, and reputational damage.

Likelihood: Medium (2)

Impact: High (3)

Risk Level: High (6)

R9 – OS Command Injection / Path Traversal

Affected Components:

`Report Module`, `Infrastructure Layer`

Related Requirements:

FR-13 (Sales report export with custom Report Name)

Description:

Improper handling of file paths or report names may allow execution of OS commands or unauthorized file access.

Attacker Objective:

Execute commands on the server or access restricted files.

Threat Impact:

May lead to system compromise, data breaches, and operational disruption.

Likelihood: Medium (2)

Impact: High (3)

Risk Level: High (6)

Security requirements

ASVS Verification Level

The Application Security Verification Standard (ASVS), maintained by OWASP, is a set of verifiable security requirements designed to support architects, developers, and testers in the definition, construction, testing, and verification of secure applications [1]. The standard is structured into three verification levels, Opportunistic (Level 1), Standard (Level 2), and Advanced (Level 3), with each level inheriting all requirements from the preceding one [1].

Based on the analysis of the Tea Shop application's risk profile and on the definitions provided in the course supporting documentation [2], it has been established that the application shall be developed in compliance with ASVS 5.0 Level 2 (Standard).

The Tea Shop falls to Level 2 for the following reasons:

1. Handling of sensitive data and financial transactions. The application processes orders with associated monetary value, maintains consistency between orders, payments, and stock, and stores customers' personal data subject to the General Data Protection Regulation (GDPR), namely name, phone contact, and billing address. Although the direct processing of card data is delegated to an External Payment Provider, such as SIBS (Sociedade Interbancária de Serviços), the application retains transactional references, callback flows, and data that may be exploited by malicious actors for financial gain, a scenario identified by ASVS as characteristic of Level 2 [2].
2. Publicly exposed attack surface. As a web-based e-commerce application accessible over the Internet, it is subject to realistic opportunistic threats, including SQL injection attacks, validation failures, and attacks targeting authentication and authorization mechanisms. ASVS Level 1 does not adequately cover this risk profile, whereas Level 2 was specifically designed for applications susceptible to this class of attacks [2].
3. Differentiated authorization model. The application provides for three types of users, customer, manager, and administrator, with distinct privileges, which introduces the risk of elevation of privilege and requires authorization controls not robustly covered by Level 1.

The adoption of Level 3 (Advanced) was considered but deemed disproportionate. This level is recommended for applications that process highly sensitive information, such as healthcare systems, defense, or critical banking systems [2]. The Tea Shop does not store card data (the External Payment Provider, typically PCI-DSS certified, assumes that responsibility), does not handle medical or classified data, and its threat model does not contemplate sophisticated attackers or Advanced Persistent Threats.

The adoption of Level 2 entails cumulative compliance with the Level 1 and Level 2 requirements of ASVS 5.0, which will be tracked through the ASVS 5.0 Tracker, with the applicability of each requirement individually justified as it can be shown on the following Table 1.

Table 1 - Applicability Analysis

Chapter	Does it Apply?	Risk Mitigation
V1- Encoding and Sanitization	Yes	R2, R4
V2- Validation and Business Logic	Yes	R3, R5
V3- Web Frontend Security	Partially	-
V4- API and Web Service	Partially	R1, R2, R6, R7
V5- File Handling	Yes	R4
V6- Authentication	Yes	R1
Session Management	Yes	R1
V8- Authorization	Yes	R1
V9- Self-contained Tokens	N/A	-
V10- OAuth and OIDC	N/A	-
V11- Cryptography	Yes	R1, R6
V12-re Communication	Yes	R5
V13- Configuration	Yes	-
V14- Data Protection	Yes	R6
V15- Secure Coding and Architecture	Yes	-
V16 -Security Logging and Error Handling	Yes	R1, R6
V17- WebRTC	N/A	-

Fully applicable chapters

Chapters V1, V2, V5, V6, V7, V8, V11, V12, V13, V14, V15, and V16 apply in full, covering core security areas such as input sanitization, business logic validation, file handling, authentication, session management, authorization, cryptography, secure communication, configuration, personal data protection, secure coding practices, and event logging. Each directly mitigates one or more of the identified risks.

Partially applicable chapters

Chapter V3 applies to the client app that consumes the REST API, namely in the requirements for HTTP headers (Content Security Policy), cookie management, and protection against Cross-Site Scripting. Input validation is performed on the backend, so the equivalent client-side requirements are complementary.

Non-applicable chapters

V9: session management relies on opaque tokens persisted in the database (Session entity), with immediate revocation via the isRevoked flag. The relevant requirements are covered by V7.
V10: authentication is internal, using email and password (FR-01, FR-02), with no recourse to federated authentication.

V17: the application does not use real-time communication; all interactions take place over conventional HTTP/HTTPS.

Requirements Mapping by Aggregate

User Aggregate

Holds personal data and credentials, making it one of the system's critical assets. The priority requirements cover authentication, credential storage, role-based authorization, and protection of PII under the GDPR.

Table 2 - ASVS requirements applicable to the User Aggregate

Requirement	Application	Risk
V6.2.1	Minimum password length of 8 characters (15 recommended), enforced at registration and change.	R1
V6.2.2	Password change functionality available in account management.	R1
V6.3.1	Rate limiting per account and per IP on the authentication endpoint.	R1
V8.2.1	Role verification on all Application Services with sensitive operations.	R1
V11.4.1	PasswordHash using bcrypt, Argon2id, or scrypt; MD5/SHA-1 forbidden.	R1
V14.1.1	Email, address, phone contact, and role classified as PII under the GDPR.	R6
V16.2.1	Logging of login attempts (success and failure), excluding passwords and hashes.	R1,R6

Session Aggregate

Manages active sessions through the Session entity, backed by opaque tokens with immediate revocation. The priority requirements cover secure token generation, server-side validation, timeouts, and reliable invalidation.

Table 3 - ASVS requirements applicable to the Session Aggregate

Requirement	Application	Risk
V7.1.1	Inactivity timeout and absolute lifetime reflected in Session.expiresAt	R1
V7.2.1	Session.isValid() executed by the AIM Middleware on all authenticated requests.	R1
V7.2.2	sessionToken generated with a CSPRNG, minimum entropy of 128 bits.	R1
V7.3.1	Inactive session requires re-login, verified via expiresAt.	R1
V7.4.1	Logout sets isRevoked = true; revoked tokens no longer validate forbidden.	R1
V11.3.1	Token generation and hashing to not use obsolete algorithms	R1
V16.2.1	Logging of session creation, expiration, and revocation with timestamps.	R1

Testing Strategy

Achieving strong security requires a direct link between requirements and testing. Defining security requirements does not produce secure applications by itself, and so it is necessary to plan from the outset how each requirement will be verified.

With the following subsections it is shown the security testing strategy adopted for the application, aligned with the ASVS requirements defined in previous

Testing Strategy

The following Table 4 displays the strategy combines, with three complementary techniques, each covering a distinct dimension of risk.

Table 4 - Adopted testing techniques

Technique	Description	What it detects
<i>SAST</i>	Static Application Security Testing: automated analysis of source code without executing it, integrated into the CI pipeline.	Insecure patterns in code: dynamic SQL, use of weak cryptography, secrets in clear text, injection via string concatenation.
<i>Unit Tests</i>	Unit tests focused on security behaviours: domain invariants, input validation, and authorization logic.	Bugs in business rules, invariant violations, failures in validation edge cases.
<i>QA Testing</i>	Manual tests conducted by the team before delivery, covering functional scenarios and abuse attempts aligned with the identified risks.	Authentication and authorization failures, IDOR, insufficient validation, unexpected behaviour under malicious input.

Test Mapping

The table below maps each applicable ASVS chapter to the testing techniques that verify it, ensuring that every security requirement has at least one associated form of verification.

Chapter	SAST	Unit	QA
<i>V1 - Encoding and Sanitization</i>	Primary	Primary	Complementary
<i>V2 - Validation and Business Logic</i>	Complementary	Primary	Primary
<i>V3 - Web Frontend Security</i>	N/A	N/A	Primary
<i>V4 - API and Web Service</i>	N/A	Complementary	Primary
<i>V5 - File Handling</i>	Primary	Primary	Primary
<i>V6 - Authentication</i>	Complementary	Primary	Primary
<i>V7 - Session Management</i>	Complementary	Primary	Primary
<i>V8 - Authorization</i>	Complementary	Primary	Primary
<i>V11 - Cryptography</i>	Primary	Primary	N/A
<i>V12 - Secure Communication</i>	N/A	N/A	Primary
<i>V13 - Configuration</i>	Primary	N/A	Complementary
<i>V14 - Data Protection</i>	Primary	Complementary	Primary
<i>V15 - Secure Coding and Architecture</i>	Primary	Complementary	N/A
<i>V16 - Security Logging and Error Handling</i>	Complementary	Primary	Complementary

Tools and Pipeline Integration

The selected tools are open-source solutions with strong support for CI/CD pipeline integration. The adopted approach follows the shift-left principle, the earlier a vulnerability is detected, the lower the cost of fixing it.

Table 5 - Tools and Purpose

Technique	Tool	Purpose	Phase
<i>SAST</i>	SonarQube	Static code analysis, detection of code smells and vulnerabilities against known patterns.	Pre-commit and CI (pull request)
<i>Unit Tests</i>	Stack's testing framework	Automated tests of domain invariants, authentication, and authorization logic.	Pre-commit and CI
<i>QA Testing</i>	Risk-driven manual checklist	Manual verification of abuse scenarios foreseen for R1–R7.	Phase 2

The pipeline includes three verification moments: pre-commit (unit tests and SAST running locally in the IDE), CI on each pull request (full SAST scan and automated test suite, with merge blocked in the presence of High or Critical vulnerabilities), and manual QA at Phase 2.

Test Priorities

Tests are prioritized according to the risk level assigned in the Risk Assessment section. High-level risks receive combined coverage from all three techniques; Medium-level risks are covered by at least two.

Table 6 - Testing strategy by risk

ID	Risk	Level	Testing strategy
R1	Authentication Bypass	High (9)	Unit tests on Session.isValid() and authorization controls; manual QA simulating bypass attempts on V6, V7, and V8.
R2	SQL Injection	High (9)	SAST with rules specific to SQL string concatenation; manual code review of repositories; QA with payloads on query endpoints.
R3	Race Condition in Stock Allocation	High (6)	Unit tests on StockLevel locking behaviour; QA with simulated concurrent execution of the checkout flow.
R4	Malicious File Upload	High (9)	Unit tests for MIME type and extension validation; manual QA with malicious payloads on upload (FR-07) and export (FR-13).
R5	Payment Status Manipulation	Medium–High (6)	SAST on the Payment API Client; QA simulating forged callbacks and signature validation.
R6	Exposure of Sensitive Data	Medium–High (6)	SAST with PII rules; code review of response DTOs; QA focused on leakage in error messages.
R7	Denial of Service	Medium	QA with load on public endpoints; manual verification of the configured rate limiting.

REFS

- [1] OWASP Foundation, *OWASP Application Security Verification Standard 5.0.0*, 2024.
- [2] P. Baltarejo Sousa, "*T4 - Secure Software Development Process*", Desenvolvimento de Software Seguro, ISEP, 2025/26.
- [3] OWASP Foundation, *OWASP Web Security Testing Guide (WSTG)*, v4.2, 2020.
- [4] NIST, *SP 800-218 Secure Software Development Framework (SSDF)*, Version 1.1, 2022.